RESEARCH ARTICLE

Doubly Nested Semantic Communication: Rate-Nested Token Transmission and Time-Nested Language-Model Memories for Drift-Resilient 6G Links

[Author names, affiliations, ORCID iDs and the corresponding author to be inserted by the authors]

Funding: [Funding sources, or "This research received no specific grant from any funding agency."]

Abstract

Large language models (LLMs) let a semantic receiver decode text from few channel symbols, because the model shared by both ends predicts what the transmitter is likely to say. After deployment the content drifts, and keeping the receiver's knowledge aligned costs air time that is rarely accounted for. This paper presents doubly nested semantic communication (DN-SC), a token-level transceiver nested in rate and in time. Every token is sent as a prefix of a 16-symbol codeword with exact per-shell power control, so that the receiver can request further symbols only for uncertain tokens; a frozen GPT-2 model is the prior of a Bayesian beam-search decoder; and three receiver memories—conversation context, a logit bias and low-rank adapter slots—adapt the prior at three timescales. Return-link requests and memory synchronization are charged in channel symbols. On a hierarchical drift stream with six seeds (AWGN, mean signal-to-noise ratio 10 dB), DN-SC reaches 90 % token accuracy with 7.08 real channel symbols per token against 9.23 for a frozen fixed-rate receiver with the same decoder: 23.3 % fewer with the requests included, lower on every seed, and 27.9 % fewer (95 % confidence interval 21.6–31.7 %) when decoded free-running. The free context memory supplies about three quarters of the memories' gain; the shipped memories repay their synchronization, relative to the context memory, only when one update serves more than about 6100 messages.

Keywords

6G | semantic communication | large language models | joint source–channel coding | incremental redundancy | continual adaptation | nested learning

1 | Introduction

The sixth generation (6G) of mobile networks is expected to support services whose value lies in the meaning of what is delivered rather than in the exact reproduction of every bit [1–4]. Exact reproduction is the problem that Shannon solved [5]; semantic communication addresses the level above it, identified by Weaver [6], by designing transmitter and receiver jointly around the information the receiver needs [7–9]. Learned joint source–channel coding has shown that such systems degrade gracefully with the

channel and can outperform separate source and channel coding at short block lengths for text [10, 11] and images [12, 13].

Large language models (LLMs) add a further resource: a strong model of what a sender is likely to say. When both ends share such a model, the receiver can combine the likelihood of the received symbols with the model's prediction of the next token and decide the most probable sentence, so that fewer channel symbols suffice for the same fidelity [14]; the context among tokens can likewise be exploited across modalities [15]. Pre-trained language models have also been used to rank the importance of words [16], and large AI models are being studied as the knowledge base of semantic transceivers [17–19].

Two costs are usually left out of this picture. The first is the cost of keeping the shared knowledge aligned. After deployment the content changes—the topic of a conversation within minutes, the vocabulary of a domain over days—and a frozen receiver degrades. The receiver cannot adapt on its own, because only the transmitter knows the source text; every update of the receiver's model must therefore be carried over the very link the semantic codec is meant to economize. Existing work adapts or updates the knowledge of semantic transceivers after deployment [20–22]; for a language-model prior updated at several timescales, however, the air time of the updates has not been set against the rate they save. The second is the cost of adapting the rate. Incremental redundancy lets a receiver ask for more symbols only when it needs them [23, 24], but every request travels on a return link, and a token-level policy makes many requests.

This article studies both costs in one system, called doubly nested semantic communication (DN-SC). The system is nested in rate: every token of the GPT-2 vocabulary [25] is mapped to a codeword of 16 real channel symbols whose leading coordinates are organized in shells, each normalized to unit power, so that any prefix is a valid transmission and sending a further shell after a prefix is exactly equivalent to having sent the longer prefix at once. The receiver decodes with the language model as its prior and requests further shells only for the tokens whose posterior confidence is below a threshold. The system is also nested in time: the receiver's prior is shaped by three memories that change at three speeds—the context of the conversation (every message, free of signaling), a logit bias in embedding space (every ten adaptation steps) and low-rank adapter slots of the language model [26] (every fifty steps, with one slot per knowledge domain). The multi-frequency organization follows the nested-learning view of models as systems of components updated at different rates [27]; in a transceiver, however, each update frequency acquires a price in channel uses, and DN-SC records that price exactly.

The evaluation is designed so that the results can contradict the method. Teacher-forced emulation, which scores each token given the true tokens before it, is cheap but optimistic; on a subset of steps the same received symbols are therefore decoded for real, free-running, and a decision rule fixed in advance determines which measurement stands. The decoder of the refinement policy was chosen on validation text only, by a rule written down before it was run, and the confirmatory comparison uses six seeds, paired bootstrap intervals and an exact seed-level test.

The contributions are as follows.

1. **A rate-nested token codebook with exact power control.** Each shell of each message is scaled to unit power, so every prefix meets the power constraint and token-level incremental redundancy is exact. Nesting costs nothing measurable at the prefix that was checked: at 8 symbols per token the nested codebook reaches a channel-only token accuracy of 0.8885 against 0.8890 for a codebook

trained for 8 symbols alone.

2. **A time-nested receiver with an exact synchronization ledger.** The context memory is built by the receiver from its own decisions and costs nothing on the air; the logit-bias and adapter memories are learned at the transmitter from the true text and shipped as quantized differences at their own periods. Adapter slots keep one adapter per domain, and every shipped bit is counted.

3. **A complete account in channel symbols.** Forward symbols, the return-link signaling of the refinement requests (as an explicit bitmap and at its entropy) and the synchronization of the memories are all converted to real channel symbols, giving a net semantic rate and the traffic volume above which shipping adaptation pays.

4. **A pre-specified evaluation and its result.** On a hierarchical drift stream with six seeds, DN-SC needs 23.3 % fewer real channel symbols per token than a frozen fixed-rate receiver with the same decoder at 90 % token accuracy, with the request signaling included, and 27.9 % fewer when the audited messages are decoded for real (the synchronization of the memories is accounted separately); the decomposition shows that the free context memory carries most of the gain of the memories and that shipped adaptation pays only above a large traffic volume.

The code, the configurations and every result file behind the tables are public (see the Data Availability Statement).

The remainder of the article is organized as follows. Section 2 reviews related work. Section 3 states the system model and the problem. Section 4 presents the DN-SC receiver, its memories, the refinement protocol and the cost accounting. Section 5 describes the experimental protocol, Section 6 reports and discusses the results, and Section 7 concludes.

2 | Related Work

2.1 | Learned Semantic Transmission of Text

Deep joint source–channel coding (JSCC) replaces the separate source and channel codes by one learned mapping from content to channel symbols. For text, Farsad et al. [10] encoded sentences with recurrent networks, and DeepSC [11] used a Transformer [28] with a fixed number of symbols per word, trained and evaluated on the Europarl corpus [29]; its restriction to sentences of 4–30 words has been adopted by many later studies. For images, deep JSCC avoids the cliff effect of separate coding [12] and can deliver progressively refinable bandwidth-agile codes [13]. Surveys and tutorials place these designs within the broader program of semantic and goal-oriented communication [2, 7–9]. Two properties of this line of work matter here. The encoder and decoder are typically trained jointly for a data distribution that is assumed fixed, and the number of channel uses per message is either fixed or chosen from a small set at the transmitter.

Rate adaptation at the receiver's request has been studied as semantic hybrid automatic repeat request (HARQ). Jiang et al. [24] combined a learned sentence codec with retransmission and incremental knowledge, following the incremental-redundancy principle of classical HARQ [23]. Successive refinement [30] and nested representations [31] supply the structural idea behind prefix codes whose leading part is useful on its own. DN-SC applies incremental redundancy per token rather than per message and

makes the request depend on the posterior confidence of the receiver's language prior; it also charges the requests on the return link, which a token-level policy makes frequent.

2.2 | Language Models in Semantic Communication

Pre-trained language models enter semantic communication in three roles. As a measure of importance, they rank words so that power or bandwidth can be allocated to those that matter most [16]. As a knowledge base, large AI models supply the background knowledge of the transceiver and are argued to be a natural foundation for 6G semantic systems [17–19]; LLM-based text transceivers have been proposed that dispense with a fixed knowledge base [32], and large models have been fine-tuned and compressed for multi-task semantic communication [33]. As a prior, finally, the language model provides the probability of the next token given what has been decoded, which the receiver combines with the channel likelihood. Wang et al. [14] built a text semantic communication system on this principle, with tokens as semantic units and a pre-trained LLM as the prior of the decoder; Qiao et al. [15] exploited the cross-modal context among tokens, through large models, to save bandwidth. DN-SC belongs to this third group and takes the LLM-prior decoder as its starting point. It adds the nested codebook with exact per-shell power control, token-level selective refinement, and memories that change after deployment.

2.3 | Knowledge Mismatch and Adaptation After Deployment

A semantic receiver works because it shares knowledge with the transmitter, and that knowledge can diverge. Zhang et al. [20] studied transceivers that must cope with a task-unaware transmitter and dynamic data. Zhang et al. [21] test whether the knowledge bases of the two ends match and adapt them with a lightweight step when they do not, and cooperative update policies keep the knowledge bases of several semantic communication pairs current while limiting the overhead of the update [22]. In machine learning, adaptation to a changing data stream raises the problem of catastrophic forgetting [34]; low-rank adaptation (LoRA) [26] makes updates of a large model small; and communication-efficient distributed learning compresses updates by quantization [35, 36]. The nested-learning paradigm [27] organizes a model as components updated at different frequencies. In a transceiver, however, the receiver cannot compute its own updates without the source text, so every update has a price in air time that grows with its size and frequency. DN-SC differs from the work above in that it charges every update of a language-model prior, at three timescales, bit by bit and in channel symbols, against the rate that the update saves; the result is the traffic volume above which updating pays.

2.4 | Positioning

Table 1 positions DN-SC against the closest work along the dimensions this article studies. The table records what each work addresses as its subject; a dash does not mean that the work could not be extended in that direction.

3 | System Model and Problem Statement

3.1 | Link, Knowledge, and Information Pattern

A transmitter, for example a base station or an edge server, sends text messages to a receiver over a wireless link. Both ends hold the same frozen pre-trained language model p_θ with vocabulary $\mathcal{V}$ ($|\mathcal{V}| = V$)

TABLE 1 | Positioning of DN-SC against representative work. ✓: addressed as a subject of the work; —: not addressed as a subject.

Work	Pre-trained language model in the link	Receiver-driven incremental redundancy	Knowledge or model adaptation after deployment	Update overhead as a design quantity	Several update timescales
Semantic HARQ [24]	—	✓	—	—	—
Importance-aware SC [16]	✓	—	—	—	—
LLM-enabled text SC [14]	✓	—	—	—	—
Token communications [15]	✓	—	—	—	—
Dynamic-data SC [20]	—	—	✓	—	—
Knowledge-base adaptation [21]	—	—	✓	—	—
Knowledge-base update [22]	—	—	✓	✓	—
DN-SC (this work)	✓	✓	✓	✓	✓

and the same semantic codebook $\{\mathbf{c}_v\}_{v\in\mathscr{V}}$, $\mathbf{c}_v \in \mathbb{R}^M$. A message is a sequence of tokens $\mathbf{x} = (x_1, \ldots, x_T)$, $x_t \in \mathscr{V}$; token x_t is sent as a prefix of $\mathbf{c}_{x_t}$ of $m \leq M$ real channel symbols, and $m/2$ complex channel uses carry it. The receiver additionally holds three memories that shape its prior (Section 4.5). The information pattern is as follows. The transmitter knows the text, the codebook, the mean signal-to-noise ratio (SNR) fed back by the receiver and its own working copies of the memories; it does not know the receiver's noise or decisions. The receiver knows its channel state, the number of tokens of each message (the number of symbol groups it receives) and its own memories. A return link carries the receiver's requests for further symbols, and the forward link carries, besides the payload, the updates of the receiver's memories.

3.2 | Traffic and Drift

Traffic is organized in adaptation steps $n = 1, 2, \ldots$. At every step each of N_c parallel conversations sends the next sentence of its current document, and a conversation that reaches the end of a document opens a new one. The content drifts at three timescales. The fast band is the channel and the document of each conversation. The medium band is the topic from which new documents are drawn; it changes every τ_{med} steps, and a new document is drawn from the current topic with probability κ and otherwise from a topic chosen uniformly at random, which may again be the current one. The slow band is the knowledge domain, which changes every τ_{slow} steps and returns after the other domains have been visited, so that retained knowledge can be measured. The SNR of step n, in decibels, follows a clipped first-order autoregressive process,

$$\tilde{\gamma}_n = \mu_\gamma + \rho\,(\tilde{\gamma}_{n-1} - \mu_\gamma) + \sigma_\gamma\sqrt{1-\rho^2}\,\varepsilon_n, \qquad \gamma_n = \min\{\gamma_{\max}, \max\{\gamma_{\min}, \tilde{\gamma}_n\}\}, \tag{1}$$

with $\varepsilon_n \sim \mathscr{N}(0,1)$ independent and $\tilde{\gamma}_0 = \mu_\gamma$; the stationary standard deviation of $\tilde{\gamma}_n$ is σ_γ.

3.3 | Channel

Each transmitted block (one shell of the codewords of the tokens of one message that a round carries, Section 4.2) is scaled to unit average power per symbol by a gain. After coherent equalization with perfect channel state information, symbol j of token t is received as

$$y_{t,j} = c_{x_t,j} + w_{t,j}, \qquad w_{t,j} \sim \mathscr{N}\left(0, \lambda_{t,j}^{-1}\right), \qquad \lambda_{t,j} = g_{t,k(j)}^2\,|h|^2\,10^{\gamma_n/10}, \tag{2}$$

where $k(j)$ is the shell that contains coordinate j, $g_{t,k}$ is the gain of the block that carried shell k of token t, $|h|^2 = 1$ for the additive white Gaussian noise (AWGN) channel and $|h|^2 \sim \mathrm{Exp}(1)$, drawn once per message and floored at $|h| = 0.05$, under Rayleigh block fading. The noise is independent across symbols, tokens and messages. A capacity-achieving code at the mean SNR $\bar{\gamma}$ carries $C(\bar{\gamma}) = \frac{1}{2}\log_2(1+\bar{\gamma})$ bits per real channel symbol, with $\bar{\gamma}$ on a linear scale ($C = 1.7297$ at 10 dB); this is the rate at which the signaling and synchronization bits are charged.

3.4 | Performance Measures and Problem Statement

Fidelity is measured by the token accuracy A, the fraction of transmitted tokens that the receiver decides correctly, with BLEU-4 [37] reported where sentences are compared. Cost is measured in real channel symbols per token, and three parts are distinguished: the forward symbols S_{fwd} of the payload, the return-link signaling B_{fb} of the refinement requests (bits per token) and the synchronization B_{sync} of the receiver's memories (bits per adaptation step). If one adaptation step serves N messages of $\bar{T}$ tokens on average, a receiver policy π—its memories, decoder and refinement rule—costs

$$S_{\mathrm{tot}}(\pi;N) = S_{\mathrm{fwd}}(\pi) + \frac{B_{\mathrm{fb}}(\pi)}{C(\bar{\gamma})} + \frac{B_{\mathrm{sync}}(\pi)}{C(\bar{\gamma})\,\bar{T}\,N}, \qquad \min_{\pi} S_{\mathrm{tot}}(\pi;N) \quad \text{subject to} \quad A(\pi) \geq \alpha^{\star}, \tag{3}$$

real channel symbols per token, where $\alpha^{\star}$ is a target token accuracy. Charging the bits at capacity and assuming the return link error-free favors every policy that signals or synchronizes, all of them in the same way. The first two terms of (3) are measured per message; the third depends on how many messages one update serves, which is a property of the deployment rather than of the codec. The experiments therefore report $S_{\mathrm{fwd}} + B_{\mathrm{fb}}/C(\bar{\gamma})$ at $\alpha^{\star} = 0.9$ as the primary endpoint and treat the synchronization term as a function of N (Section 4.7).

4 | Doubly Nested Semantic Communication

4.1 | Overview

Figure 1 shows the transceiver. The transmitter tokenizes a message with the tokenizer of the language model, maps every token to its codeword in the rate-nested codebook and transmits the codewords shell by shell, each shell of each message at exactly unit power. The receiver computes, for every token position, the likelihood of every vocabulary entry from the symbols received so far, combines it with the prior of the language model and its memories, and decodes the message by beam search. Tokens whose posterior confidence stays below a threshold are requested again, and the next shell is sent for them only. The receiver's prior depends on three memories, which change at three speeds; the two that need the true text are learned at the transmitter and shipped as quantized differences at their own periods. The remainder of this section specifies each part.

4.2 | Rate-Nested Semantic Codebook

Codewords. Let $\mathbf{e}_v \in \mathbb{R}^D$ be the input embedding of token v in the frozen language model and s_e the standard deviation of all entries of the embedding table. A small network maps the scaled embedding

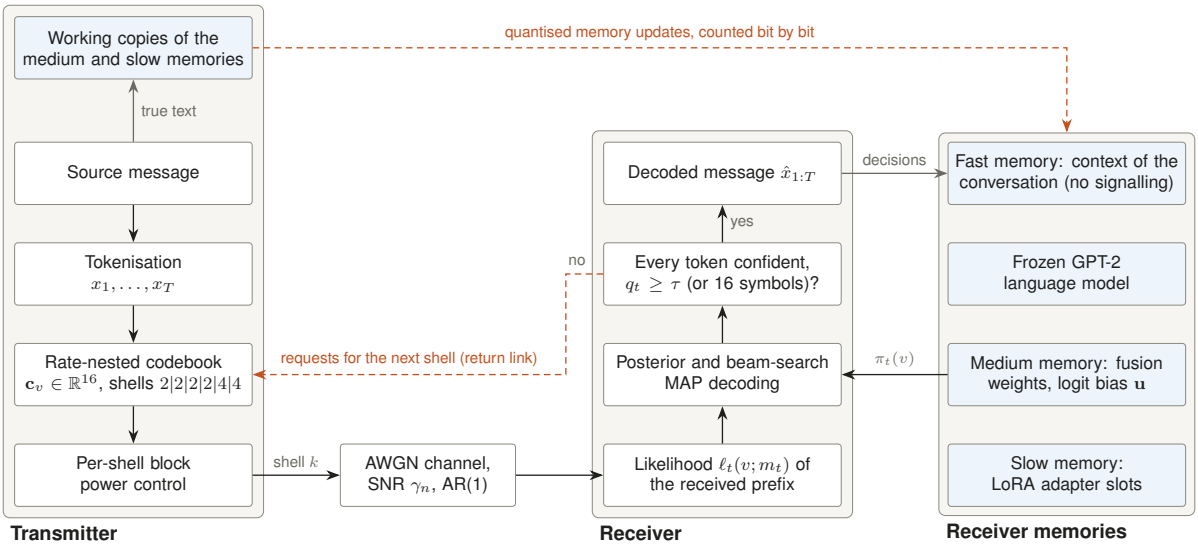


FIGURE 1 | Architecture of DN-SC. Solid arrows: forward data path. Dashed arrows: the return-link requests for further shells and the shipments that synchronize the receiver's medium and slow memories; both are charged in channel symbols. The fast memory (context) is built by the receiver from its own decisions and costs no signaling.

$\bar{\mathbf{e}}_v = \mathbf{e}_v / s_e$ to a raw codeword,

$$\tilde{\mathbf{c}}_v = \mathbf{W}_1 \bar{\mathbf{e}}_v + \mathbf{W}_3 \, \mathrm{GELU}(\mathbf{W}_2 \bar{\mathbf{e}}_v + \mathbf{b}_2), \tag{4}$$

with $\mathbf{W}_1 \in \mathbb{R}^{M \times D}$, $\mathbf{W}_2 \in \mathbb{R}^{H \times D}$, $\mathbf{b}_2 \in \mathbb{R}^H$ and $\mathbf{W}_3 \in \mathbb{R}^{M \times H}$; $\mathbf{W}_3$ is initialized at zero, so training starts from a linear projection of the embeddings. No language model runs at the transmitter to encode.

Shells. The M coordinates are cut into K shells at the prefix lengths $m_1 < m_2 < \cdots < m_K = M$; shell k holds the coordinates $\mathscr{S}_k = \{m_{k-1}+1, \ldots, m_k\}$ ($m_0 = 0$) and has width $d_k = m_k - m_{k-1}$. Each shell is centered and scaled under the unigram distribution q of the base corpus, and the energy of every codeword in it is limited to η times the average:

$$\hat{\mathbf{c}}_{v,k} = \frac{\tilde{\mathbf{c}}_{v,\mathscr{S}_k} - \mu_k}{\sqrt{P_k}}, \qquad \mathbf{c}_{v,\mathscr{S}_k} = \min\left\{1, \frac{\sqrt{\eta\, d_k}}{\|\hat{\mathbf{c}}_{v,k}\|}\right\} \hat{\mathbf{c}}_{v,k}, \tag{5}$$

where $\mu_k = \sum_v q(v)\, \tilde{\mathbf{c}}_{v,\mathscr{S}_k}$ and $P_k = d_k^{-1} \sum_v q(v)\, \|\tilde{\mathbf{c}}_{v,\mathscr{S}_k} - \mu_k\|^2$. Every shell therefore has zero mean before the peak limit and average power at most one per symbol under q.

Block power control. The constraint of (5) holds on average over the base corpus, not for a particular message. Before transmission, shell k of the set $\mathscr{T}$ of tokens of a message that a round carries—all T tokens in the first round, the requested ones in a refinement round—is therefore multiplied by

$$g_k(\mathscr{T}) = \left(\frac{|\mathscr{T}|\, d_k}{\sum_{t \in \mathscr{T}} \|\mathbf{c}_{x_t,\mathscr{S}_k}\|^2} \right)^{1/2}, \tag{6}$$

so that every transmitted block has an average power of exactly one per symbol. The gain is applied to the whole block, pilots included, and is absorbed in the channel estimate; the receiver sees it only as the precision $\lambda_{t,j}$ of (2). As a consequence, no receiver can gain power from content whose codewords happen to be large.

Property 1 (exact incremental refinement). Every transmitted block meets the power constraint with

equality, so every prefix does. Receiving shells $1,\dots,k$ of all tokens in k separate rounds yields the same observations, in distribution, as receiving the prefix m_k at once; a refinement round extends the prefix of each requested token by one shell, received with the precision set by the gain of the requested set. *Proof.* The gain (6) of a block depends only on the coordinates of its own shell and on the tokens it carries, and the noise in (2) is independent across symbols; hence neither a block nor its noise depends on when, or whether, other shells are sent. Averaging the unit power of the blocks gives the power of any prefix. □

4.3 | Bayesian Receiver With a Language Prior

Likelihood. When the first m symbols of token t have arrived, the Gaussian log-likelihood of every vocabulary entry is, up to a term that does not depend on v,

$$\ell_t(v;m) = \sum_{j=1}^{m} \lambda_{t,j} \left(y_{t,j}\, c_{v,j} - \tfrac{1}{2}\, c_{v,j}^2 \right), \tag{7}$$

which is one matrix product of the received symbols with the codebook for all V entries.

Prior. Let $\hat{\mathbf{x}}_{<t}$ be the tokens already decided in the message, $\mathbf{h}$ the context (Section 4.5) and ϕ_s the active adapter slot ($\phi_\varnothing$: none). The prior of the receiver is

$$\log \pi_t(v) = \log \mathrm{softmax}_v \left[w_{\mathrm{p}} \log p_\theta(v \mid \mathbf{h}, \hat{\mathbf{x}}_{<t}; \phi_s) + \frac{1}{s_e}\, \mathbf{e}_v^\top \mathbf{u} \right], \tag{8}$$

where w_{p} sharpens or flattens the language model and $\mathbf{u} \in \mathbb{R}^D$ defines a logit bias in embedding space, which moves the probability of all tokens that are close in meaning together with one vector.

Posterior, decision and confidence. The posterior of token t after m_t symbols, its decision and the receiver's confidence in the decision are

$$p_t(v) = \frac{\exp[\log \pi_t(v) + w_{\mathrm{l}}\, \ell_t(v;m_t)]}{\sum_{v' \in \mathscr{V}} \exp[\log \pi_t(v') + w_{\mathrm{l}}\, \ell_t(v';m_t)]}, \qquad \hat{x}_t = \arg\max_v\, p_t(v), \qquad q_t = \max_v\, p_t(v), \tag{9}$$

with a likelihood weight w_{l} ($w_{\mathrm{p}} = w_{\mathrm{l}} = 1$ and $\mathbf{u} = \mathbf{0}$ before any adaptation).

Beam search. Decoding is free-running: the language model is conditioned on the receiver's own earlier decisions, so an early error misleads the prior for the rest of the sentence. The receiver therefore keeps the B best partial sentences at every position, ranked by their joint log-probability with the received symbols,

$$J(x_{1:t}) = \sum_{i=1}^{t} \left[\log \pi_i(x_i \mid x_{<i}) + w_{\mathrm{l}}\, \ell_i(x_i;m_i) \right], \tag{10}$$

which equals the log-posterior of the partial sentence up to a constant. Summing the per-position posteriors (9) instead would subtract $\log p(y_i \mid x_{<i})$ at every position and so penalize exactly the prefixes that explain the next observation well, a form of the label-bias problem [38]. The confidence q_t of a decision is that of the posterior at position t on the selected path.

4.4 | Training of the Codebook

The codebook is trained once, before deployment and without the language model, by maximizing the channel-only posterior of the transmitted token at randomly drawn prefix lengths and SNRs:

$$\mathscr{L}_{\mathrm{cb}}(\mathbf{W}) = \mathbb{E}_{v\sim\bar{q},\,\gamma\sim\mathscr{U}[\gamma_{\mathrm{lo}},\gamma_{\mathrm{hi}}]}\left[\frac{1}{|\mathscr{M}_b|}\sum_{m\in\mathscr{M}_b} -\log\frac{\exp\ell(v;m)}{\sum_{v'\in\mathscr{V}}\exp\ell(v';m)}\right], \tag{11}$$

where ℓ is (7) with $\lambda_{t,j} = 10^{\gamma/10}$, $\mathscr{M}_b$ is a set of prefix lengths drawn afresh for every batch, and $\bar{q}$ mixes the unigram distribution of the base corpus with the uniform distribution over $\mathscr{V}$, so that tokens the base corpus never uses—the vocabulary of later domains—are separated as well. The objective trains all prefixes jointly, which is what makes every prefix a usable code.

4.5 | Time-Nested Receiver Memories

The prior (8) is shaped by three memories (Figure 2b).

Fast memory: context. A message that continues a document is decoded with the receiver's own reconstruction of the preceding messages of the same conversation as context $\mathbf{h}$, up to a fixed number of tokens. This is the native memory of the language model; it changes with every message, needs no gradient and costs nothing on the air. Its errors propagate, since a wrongly decoded message becomes a wrong context.

Medium and slow memories. The medium memory $\phi_{\mathrm{med}} = (w_{\mathrm{p}}, w_{\mathrm{l}}, \mathbf{u})$ and the slow memory, a set of K_{s} LoRA adapters $\phi_1,\ldots,\phi_{K_{\mathrm{s}}}$ of the language model [26], can only be learned with the true text. They are therefore learned at the transmitter, which keeps a working copy ϕ^{tx} of both and, at every adaptation step n, takes one optimizer step on

$$\mathscr{L}_n(\phi) = \frac{1}{|\mathscr{M}_n|}\sum_{m\in\mathscr{M}_n}\frac{1}{N_n}\sum_{(i,t)} -\log p_t^{\phi}\left(x_{i,t}\mid \mathbf{y}_{i,t,1:m},\,\mathbf{x}_{i,<t},\,\mathbf{h}_i^{\mathrm{tx}}\right), \tag{12}$$

the posterior cross-entropy (9) of the N_n tokens of the step's messages, with the true earlier tokens and the true context $\mathbf{h}_i^{\mathrm{tx}}$, averaged over the operating prefix length and one further prefix length drawn at random ($\mathscr{M}_n$), so that the prior serves every rate. Only the active adapter slot learns.

Quantized shipment. The receiver holds its own copy ϕ_ℓ^{rx} of level $\ell\in\{\mathrm{med},\mathrm{slow}\}$, which changes only at the level's period C_ℓ. When $n\equiv 0 \pmod{C_\ell}$, the transmitter quantizes the difference of every parameter tensor $\blacksquare = \phi_\ell^{\mathrm{tx}} - \phi_\ell^{\mathrm{rx}}$ with b bits per entry, both ends apply the quantized difference, and the cost is recorded:

$$\begin{aligned} &\mathscr{Q}_b(\blacksquare) = \frac{a}{L}\,\mathrm{round}\left(\frac{L\blacksquare}{a}\right), \qquad a = \|\blacksquare\|_\infty, \qquad L = 2^{b-1}-1, \\ &\phi_\ell^{\mathrm{rx}} \leftarrow \phi_\ell^{\mathrm{rx}} + \mathscr{Q}_b(\blacksquare), \qquad \phi_\ell^{\mathrm{tx}} \leftarrow \phi_\ell^{\mathrm{rx}}, \qquad B_\ell = \sum_{i\in\ell}(32 + b\,n_i), \end{aligned} \tag{13}$$

where the sum runs over the tensors of the level, n_i is the number of entries of tensor i and 32 bits carry its scale a (an all-zero difference costs one flag bit). Right after a shipment the two copies are bit-identical; in between, the receiver decodes with a stale copy. The period C_ℓ thus trades staleness against air time, and nothing else: all receivers learn in the same way and differ only in when each level reaches the receiver. The multi-frequency organization follows the nested-learning view [27]; its

Algorithm 1 One adaptation step n of DN-SC

Input: messages $\mathbf{x}_1, \ldots, \mathbf{x}_{N_c}$ of the step; SNR γ_n; receiver copies ϕ^{rx}, contexts $\mathbf{h}_i^{\mathrm{rx}}$; transmitter copies ϕ^{tx}, contexts $\mathbf{h}_i^{\mathrm{tx}}$
Output: decoded messages $\hat{\mathbf{x}}_i$; updated memories; bit ledger

if $(n-1) \bmod C_{\mathrm{sel}} = 0$ **then** ▷ slot choice, Eq. (14)
  transmitter evaluates H_s under ϕ^{rx}, chooses s_{new} and signals it; ledger $+= \lceil \log_2(K_{\mathrm{s}}+1) \rceil$
end if
for each message i **do** ▷ transmission and decoding
  transmitter encodes $\mathbf{x}_i$ by Eqs. (4)–(6) and sends the first shell(s), m_{k_0} symbols per token
  repeat
    receiver decodes by beam search, Eqs. (7)–(10), with prior ϕ^{rx} and context $\mathbf{h}_i^{\mathrm{rx}}$
    receiver closes every open token with $q_t \geq \tau$ and requests the next shell for the others ▷ Eqs. (15), (16)
    transmitter sends the next shell of the requested tokens
  until no token is open
  $\mathbf{h}_i^{\mathrm{rx}} \leftarrow \mathbf{h}_i^{\mathrm{rx}} \| \hat{\mathbf{x}}_i$; $\mathbf{h}_i^{\mathrm{tx}} \leftarrow \mathbf{h}_i^{\mathrm{tx}} \| \mathbf{x}_i$ ▷ fast memory
end for
transmitter takes one optimizer step on $\mathscr{L}_n(\phi^{\mathrm{tx}})$, Eq. (12) ▷ medium and active slow memory
for $\ell \in \{\mathrm{med}, \mathrm{slow}\}$ with $n \equiv 0 \pmod{C_\ell}$ **do**
  ship $\mathscr{Q}_b(\phi_\ell^{\mathrm{tx}} - \phi_\ell^{\mathrm{rx}})$ and set both copies equal; ledger $+= B_\ell$ ▷ Eq. (13)
end for

quantized, bit-counted synchronization follows the practice of communication-efficient learning [35, 36].

Adapter slots. Adapting one adapter to one domain can damage the prior for another (Section 6.6 measures the effect), so the slow memory keeps one adapter per domain. Every C_{sel} steps the transmitter evaluates, on the messages it is about to send, the prior cross-entropy H_s (bits per token) of the receiver's copy with every slot $s \in \{\varnothing, 1, \ldots, K_{\mathrm{s}}\}$, and changes the active slot s_{cur} only if another choice is better by a margin δ:

$$s^\star = \arg\min_s H_s, \qquad s_{\mathrm{new}} = \begin{cases} s^\star, & H_{s^\star} < H_{s_{\mathrm{cur}}} - \delta, \\ s_{\mathrm{cur}}, & \text{otherwise}, \end{cases} \tag{14}$$

where an untrained slot is started instead whenever $s_{\mathrm{new}} = \varnothing$ and one is left. The choice costs $\lceil \log_2(K_{\mathrm{s}}+1) \rceil$ bits. A domain that returns finds its adapter intact, so recalling it costs two bits rather than a new synchronization, and the receiver keeps an adapter that is worse than none by more than δ for at most C_{sel} steps.

Algorithm 1 summarizes one adaptation step.

4.6 | Selective Refinement and Its Signaling

Protocol. Round 1 delivers the first m_{k_0} symbols of every token. After every round the receiver decodes the whole message again with the symbols received so far and, for every token that is still open, decides whether to request its next shell; a token that is not continued is closed for good, and nothing has to be said after the last shell (Figure 2a). Under this rule, token t stops at the shell

$$k_t(\tau) = \min\left\{ k \in \{k_0, \ldots, K\} : q_t^{(k)} \geq \tau \right\} \text{ (or } K \text{ if no such } k), \qquad S_{\mathrm{fwd}}(\tau) = \frac{1}{N_{\mathrm{tok}}} \sum_t m_{k_t(\tau)}, \tag{15}$$

where $q_t^{(k)}$ is the confidence of token t when the message is decoded with m_k symbols for that token and S_{fwd} is the mean number of forward symbols per token over the N_{tok} tokens of an adaptation step; operating curves average these step means over the steps. Because the shells are nested (Property 1),

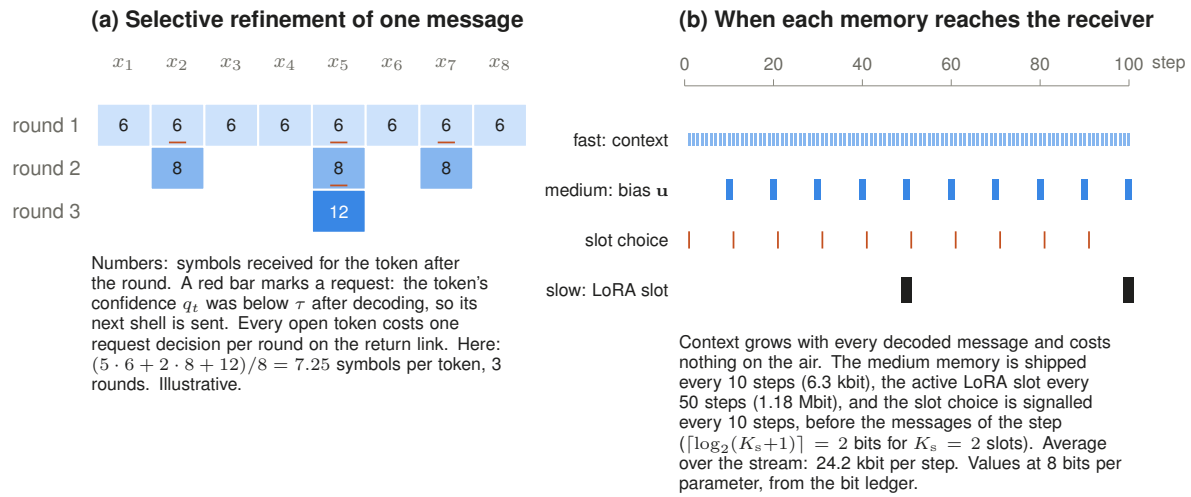


FIGURE 2 | (a) Selective refinement of one message of eight tokens with a first round of 6 symbols per token (illustrative). (b) When each memory reaches the receiver during the first 100 adaptation steps, with the costs of one shipment; the average over the stream is taken from the bit ledger of the experiments.

the extra symbols simply extend what was received: this is incremental redundancy at the level of single tokens, spent where the language model cannot fill the gap on its own.

Signaling. Let n_r be the number of tokens that are open after round r and must be decided on, and $c_r \le n_r$ the number of them that are continued. Two costs of the requests are derived from the decisions actually taken, in bits per token: an explicit bitmap, and the empirical entropy of the decisions of each round, which an ideal entropy coder of the bitmap would approach,

$$B_{\mathrm{bm}} = \frac{1}{N_{\mathrm{tok}}} \sum_r n_r, \qquad B_{\mathrm{H}} = \frac{1}{N_{\mathrm{tok}}} \sum_r n_r \, h_2\!\left(\frac{c_r}{n_r}\right), \qquad h_2(p) = -p \log_2 p - (1-p)\log_2(1-p). \tag{16}$$

Both are converted to real channel symbols at $C(\bar{\gamma})$ and added to the forward symbols. B_{H} is a lower reference, not an implemented code; the bitmap is implementable as it stands. A fixed-prefix receiver needs no signaling at all.

4.7 | Net Semantic Rate and Crossover Traffic

Let $R = \bar{T}\,[S_{\mathrm{fwd}} + B_{\mathrm{fb}}/C(\bar{\gamma})]$ be the payload of a receiver per message at the target accuracy, read off its measured operating curve (symbols against accuracy, over the thresholds τ or the prefix lengths) by linear interpolation. When even the cheapest point of a curve exceeds the target, the curve is read at that point, which is then an upper bound of the cost at the target; Section 6.3 states where this occurs. If one adaptation step, with B_{sync} bits of synchronization, serves N messages, the net semantic rate and the traffic above which the receiver is cheaper than a reference receiver with payload $R^{\mathrm{ref}} > R$ are

$$R_{\mathrm{net}}(N) = R + \frac{B_{\mathrm{sync}}}{C(\bar{\gamma})\,N} \quad \text{real symbols per message}, \qquad N^{\star} = \frac{B_{\mathrm{sync}}}{C(\bar{\gamma})\,(R^{\mathrm{ref}} - R)}. \tag{17}$$

R_{net} is (3) multiplied by $\bar{T}$. Both quantities are derived from measured payloads and a measured ledger; they are reported as such.

4.8 | Complexity

The receiver evaluates (7) for all V entries with one matrix product of cost $O(Vm)$ per token, and runs the language model once per token and beam in every decoding pass; selective refinement repeats the pass for the messages that still have open tokens, so the number of passes per message equals the number of rounds. The transmitter runs one forward and one backward pass of the language model per adaptation step, the backward pass through the adapted layers only, and every C_{sel} steps one forward pass per candidate slot of (14). The codebook is trained once and never shipped. Section 6.8 reports the measured parameter counts, latencies and run times.

5 | Experimental Setup

5.1 | Language Model, Codebook, and Corpora

The frozen prior is GPT-2 [25] with 124,439,808 parameters, a vocabulary of $V = 50{,}257$ tokens and $D = 768$. The codebook has $M = 16$ real symbols per token, nested at the prefix lengths $\{2,4,6,8,12,16\}$ (shell widths 2|2|2|2|4|4), with $H = 256$ hidden units and peak ratio $\eta = 4$; it has 213,248 parameters. It was trained once on the base corpus, the Europarl sample distributed with NLTK [29] (14,006 sentences of 4–30 words, split 80/10/10 by document), for 2000 steps of 256 tokens, a quarter of them drawn uniformly from the vocabulary, with one SNR per token drawn uniformly in $[0,20]$ dB and three prefix lengths per batch, using AdamW with a learning rate of 2×10^{-3}, 100 warm-up steps, cosine decay and gradient clipping at 1.0 (2194 s on a CPU). The drift stream uses AG News [39] (news; four topic classes) and TeleQnA [40] (telecommunication questions on standards and research; five categories), with sentences of 4–30 words. An AG News document is a title and its description; a TeleQnA document is a question, its explanation and its answer. The stream draws its documents from the training split of each corpus and the validation set of Section 5.4 from the validation split, so no sentence of the validation set occurs in the stream (within a run, documents are drawn with replacement).

5.2 | Stream, Channel, and Memories

The hierarchical stream visits AG News, TeleQnA, AG News and TeleQnA for $\tau_{\text{slow}} = 150$ steps each (600 steps), with $N_{\text{c}} = 6$ parallel conversations, a topic change every $\tau_{\text{med}} = 30$ steps and $\kappa = 0.9$; a domain change ends every conversation. One run therefore carries 3600 messages of about 18 tokens. The channel is AWGN with the SNR process (1) at $\mu_\gamma = 10$ dB, $\rho = 0.95$, $\sigma_\gamma = 4$ dB, clipped to $[0,20]$ dB; signaling and synchronization bits are charged at $C(10\,\text{dB}) = 1.7297$ bits per real symbol. The context holds up to 48 tokens of the preceding messages of a conversation. The medium memory (two fusion weights and $\mathbf{u} \in \mathbb{R}^{768}$) learns with Adam at 10^{-3} and is shipped every $C_{\text{med}} = 10$ steps; the slow memory has $K_{\text{s}} = 2$ LoRA slots of rank 8 ($\alpha = 16$) on the query–key–value projection of the last six of the twelve layers, 147,456 parameters per slot, learns with Adam at 5×10^{-4} and is shipped every $C_{\text{slow}} = 50$ steps; the slot is chosen every $C_{\text{sel}} = 10$ steps with $\delta = 0.05$ bits per token. Shipments use $b = 8$ bits per entry, and the loss (12) uses the prefix length of 6 symbols and one further prefix length per step. Table 2 collects the parameters.

TABLE 2 | Simulation parameters.

Group	Parameter	Value
Language prior	Model; parameters	GPT-2 (frozen); 124,439,808
	Vocabulary V; embedding width D	50,257; 768
Codebook	Symbols per token M; prefix lengths m_k	16; 2, 4, 6, 8, 12, 16
	Hidden units H; peak ratio η; parameters	256; 4; 213,248
	Training steps; tokens per batch; uniform share	2000; 256; 25 %
	Training SNR	$\mathscr{U}[0,20]$ dB per token
Stream	Domains, in order	AG News, TeleQnA, AG News, TeleQnA
	τ_{slow}; τ_{med}; κ; N_{c}; steps	150; 30; 0.9; 6; 600
Channel	Type; μ_γ; ρ; σ_γ; range	AWGN; 10 dB; 0.95; 4 dB; 0–20 dB
Fast memory	Context length	48 tokens
Medium memory	Content; learning rate; period C_{med}	w_{p}, w_{l}, $\mathbf{u} \in \mathbb{R}^{768}$; 10^{-3}; 10 steps
Slow memory	Slots K_{s}; LoRA rank; α; layers	2; 8; 16; last 6 of 12
	Parameters per slot; learning rate; period C_{slow}	147,456; 5×10^{-4}; 50 steps
Slot choice	Period C_{sel}; margin δ; cost	10 steps; 0.05 bits/token; 2 bits
Synchronization	Bits per entry b; scale per tensor	8; 32 bits
Decoder	Beam width B; first round m_{k_0}; reopening	4; 6 symbols; no
Refinement	Thresholds τ	0.5, 0.7, 0.9, 0.95, 0.99
Endpoint	Target token accuracy $\alpha^\star$	0.90

5.3 | Receivers

Three receivers are compared on the same streams, codebook and decoder: the *frozen* receiver, which decodes every message on its own with the unadapted prior (the per-message receiver of the literature); the *context memory* receiver, which adds the fast memory only and therefore costs no synchronization; and *DN-SC*, with all three memories. Each is evaluated at every fixed prefix length and with selective refinement. The reference of every relative change is the frozen receiver at a fixed prefix, which needs neither signaling nor synchronization. Component ablations—single-rate shipping at about the same synchronization budget, elastic weight consolidation [34], shipping after every step, and DN-SC without slots, without context or with one period for both shipped memories—come from the pilot runs of an earlier code version and are reported as exploratory (Section 6.6).

5.4 | Decoder of Selective Refinement

The decoder was chosen on validation text only, by a rule written into the repository before the selection was run. The candidates were the first round (2, 4 or 6 symbols per token), reopening of closed tokens (allowed or not) and the decoding of every pass (greedy, or beam search of width 2 or 4). The validation set holds one sentence, at a random position and with the true preceding sentences of its document as context, from each of 32 documents of the validation splits of AG News, TeleQnA and a public mirror of the News Category Dataset [41]: 96 messages and 1633 tokens, with the frozen prior, one AWGN realization per message and the SNR drawn from the stationary law of the SNR process (mean of the draws 9.8 dB). The criterion is the number of real symbols per token for 90 % token accuracy, decoded for real, with the signaling charged at its entropy. Stage 1 compares the six greedy candidates, with candidates within 1 % counted as tied and the tie going to the one with fewer rounds per message; stage 2 adds beam widths 2 and 4 to the winner, and a beam is adopted only if it lowers the criterion by more than 2 %, since it costs computation. A beam search changes the receiver, so the fixed-prefix reference is decoded with the same beam.

5.5 | Measurement Protocol

Every message is scored prequentially, with the parameters the receiver holds before the update of that step. Two measurements are taken. *Teacher-forced emulation* on every step scores each token given the true tokens before it, which needs one pass of the language model per step and yields the complete operating curve (all prefix lengths, all thresholds); the decisions of the emulated policy at $\tau = 0.9$ are what the receiver writes into its context. *Real decoding* on every 23rd step (26 steps, 156 messages per run; 23 is a prime, so the audit is never in phase with a shipping period) decodes the same received symbols free-running, round by round and with the configured decoder, for selective refinement at every threshold and for every fixed prefix alike. A decision rule fixed in advance determines which measurement is the result: the teacher-forced curves stand if, on the audited steps, real decoding needs at most 5 % more forward symbols at the target than the emulation for every receiver of the headline table; otherwise the real-decoding figures replace them on both sides of every comparison. As implemented, the check compares the refinement policy of every receiver; Section 6.3 reports it for the fixed-prefix receivers as well. In the implementation, the adaptation loss (12) of a step reuses that step's channel realization; since an update reaches the receiver only after the step has been decoded, and the noise is independent across steps, this is equivalent in distribution to a realization drawn by the transmitter. Every message is transmitted once, with the gains of (6) computed over all its tokens, and the refinement rounds, emulated and real alike, are read off that transmission; a refinement round therefore carries its shell at the gain of the whole message rather than at the gain of the requested tokens. Section 6.4 measures what this simplification changes.

5.6 | Statistics and Runs

The frozen receiver and DN-SC were run on seeds 0–5 and the context memory receiver on seeds 0, 3, 4 and 5; its runs on seeds 1 and 2 were stopped by a restart of the machine and not repeated. Seeds 3–5 had never been run before the protocol was fixed. Seeds 0–2 ran on a Linux server CPU with one thread per run and seeds 3–5 on a Windows 11 laptop CPU with two threads per run, from code revisions whose stream runner and decoder are identical. The seed is the unit of replication. For real decoding, the operating curves of the audited steps are pooled over the seeds, every receiver is compared with the frozen receiver on the same seeds and steps, and the 95 % interval of each relative change is the percentile interval of 2000 paired bootstrap draws [42] of the audited steps within each seed, a draw in which a curve misses the target counting as an infinite cost. The seed-level test is an exact sign-flip permutation test on the per-seed costs; with six seeds its smallest attainable two-sided p-value is $2/2^6 = 0.031$, reached only when all seeds agree in sign, and with four seeds it is 0.125. The tests are Holm-corrected [43] over the receivers compared with the frozen reference. Differences in prior cross-entropy between receivers are given with 95 % block-bootstrap intervals over steps (blocks of 25 steps) [44], and a claim of no meaningful difference requires two one-sided tests [45] with the 90 % interval inside ± 0.05 bits per token.

TABLE 3 | Free-running token accuracy and BLEU-4 on the base corpus (200 test sentences, AWGN, 10 dB, greedy decoding) for four receiver priors, against the number of real channel symbols per token m.

	Token accuracy				BLEU-4			
Receiver prior	$m = 4$	$m = 6$	$m = 8$	$m = 12$	$m = 4$	$m = 6$	$m = 8$	$m = 12$
None (maximum likelihood)	0.3728	0.7032	0.9119	0.9980	0.0354	0.3813	0.7672	0.9946
Unigram	0.4081	0.7366	0.9291	0.9985	0.0506	0.4261	0.8152	0.9971
GPT-2	0.4633	0.8570	0.9706	0.9995	0.1839	0.7059	0.9273	0.9990
GPT-2 + true history (diagnostic)	0.5510	0.8930	0.9818	0.9995	0.3103	0.7819	0.9607	0.9990

6 | Results and Discussion

6.1 | Link-Level Operating Point on the Base Corpus

Table 3 gives the free-running token accuracy and BLEU-4 of the codec on 200 test sentences of the base corpus over AWGN at 10 dB, for four receiver priors and a greedy decoder. The language prior lowers the rate needed for a given fidelity at prefix lengths of 4 to 12 symbols (at 2 symbols per token GPT-2 is slightly below the other priors, and at 16 all are essentially error-free): at 6 symbols per token GPT-2 decides 85.7 % of the tokens correctly, against 73.7 % with token frequencies alone and 70.3 % with no prior, and BLEU-4 rises from 0.38 (no prior) to 0.71. The last row is a diagnostic that gives the receiver the true preceding sentences of the document and bounds what the fast memory can contribute (89.3 % at 6 symbols). Under Rayleigh block fading at 10 dB and 8 symbols per token the four priors reach 0.681 (none), 0.731 (unigram), 0.758 (GPT-2) and 0.785 (GPT-2 with true history). At 0 dB the order reverses under both channels: GPT-2 falls below the unigram prior at every prefix length (at 8 symbols per token, 0.119 against 0.196 over AWGN and 0.146 against 0.199 under Rayleigh fading), most likely because the greedy decoder then follows its own early errors. The operating point of this study is therefore an SNR around 10 dB.

Rate nesting costs nothing measurable at the prefix that was checked: a codebook trained with the same budget for 8 symbols per token only reaches a channel-only token accuracy of 0.8890 at 10 dB on 2000 held-out tokens, and the nested codebook, read at its 8-symbol prefix, reaches 0.8885. Other prefix lengths were not tested. Before block power control, the codewords of the base corpus have an average power of 0.81–0.89 per symbol over the six prefixes; block power control raises every transmitted block to exactly one.

6.2 | Decoder of Selective Refinement

Table 4 reports the selection on validation text. In stage 1, a first round of 4 symbols has the lowest criterion (8.652 symbols per token), and a first round of 6 symbols (8.730) is within 1 % of it and needs fewer rounds (3.01 against 4.01 per message), so it wins under the rule. Reopening loses in every case once its signaling is charged: every token with shells left must take part in every round's decision, and at the lowest threshold its requests cost 2.3–4.2 bits per token at the entropy against 1.5–2.5 without reopening. In stage 2, beam search of width 4 lowers the criterion by 4.9 %, above the 2 % bar, whereas width 2 (1.5 %) does not. For six of the eight candidates, the selected one included, the operating curve exceeds 90 % already at the lowest threshold of the validation grid ($\tau = 0.7$), so the criterion is read at that point and is an upper bound (the selected decoder reaches 94.5 % there, the greedy decoder with the same

TABLE 4 | Selection of the decoder on validation text (96 messages, 1633 tokens, mean SNR 9.8 dB): real channel symbols per token for 90 % token accuracy, decoded for real. Selective refinement is charged its signaling at the entropy (criterion) or as a bitmap; the fixed prefix is decoded with the same beam width. Where a curve exceeds 90 % at its lowest threshold ($\tau = 0.7$; all rows but the first and third), the value is read there and is an upper bound. Rounds per message at confidence 0.95.

Beam	First round (symbols)	Reopening	Forward only	+ signaling (entropy)	+ signaling (bitmap)	Fixed prefix, same beam	Rounds per message
1	2	no	7.527	8.971	9.442	10.994	5.00
1	2	yes	7.486	9.937	10.541	10.994	5.34
1	4	no	7.395	8.652	8.740	10.994	4.01
1	4	yes	7.682	9.640	10.060	10.994	4.22
1	6	no	7.836	8.730	8.758	10.994	3.01
1	6	yes	8.211	9.533	9.948	10.994	3.20
2	6	no	7.738	8.603	8.637	10.044	3.01
4	**6**	**no**	**7.492**	**8.306**	**8.357**	**9.049**	**3.01**

first round 91.8 %). The measured points support the decisions: at their cheapest points beam 4 is both cheaper and more accurate than the stage-1 winner, and the bound of the 6-symbol first round can only fall, so that candidate ties with or beats the 4-symbol one. The selected decoder—beam width 4, a first round of 6 symbols, no reopening—needs 7.4 % fewer symbols than the greedy decoder from 2 symbols. The selection also shows that beam search helps the fixed prefix far more than it helps refinement: with width 4 the fixed prefix needs 17.7 % fewer symbols than with greedy decoding (9.049 against 10.994), because the beam removes much of the error propagation that refinement had been compensating for. Every fixed-prefix reference below is decoded with the same beam, so the comparisons are made against this stronger baseline.

6.3 | Primary Endpoint on the Drift Stream

Table 5 and Figure 3 give the primary endpoint. With its memories and selective refinement, DN-SC reaches 90 % token accuracy with 7.08 real channel symbols per token, against 9.23 for the frozen fixed-rate receiver: a saving of 23.3 % with the return-link signaling charged at its entropy (28.3 % for the forward symbols alone and 20.7 % with an explicit bitmap). The saving is observed on every seed, between 10.5 % and 28.5 % (exact sign-flip test $p = 0.031$, the smallest value six seeds allow; Holm-adjusted over the three refinement receivers, $p = 0.094$), and on the three seeds that had never been run before (20.6 %). Decoded for real on the audited steps, the saving is 27.9 % (95 % CI 21.6–31.7 %; 26.3 % with a bitmap), again on every seed (Holm-adjusted $p = 0.094$), and 25.9 % (95 % CI 11.0–33.2 %) on the unseen seeds alone. Seed 3 is the weakest on both measures (-10.5 % teacher-forced, -9.1 % decoded for real). These figures do not include the synchronization of the shipped memories, which is charged separately in Section 6.7. They are conservative for DN-SC: its operating curve already exceeds 90 % at the cheapest point measured ($\tau = 0.5$) on four of the six seeds teacher-forced (90.3–93.1 %) and on the pooled curve decoded for real (92.1 %), so its cost is read at that point, an upper bound of the cost at 90 %. The same holds for the context memory receiver on three of its four seeds and on its pooled real-decoding curve (90.3 %), and for the frozen receiver with refinement on one seed; the frozen fixed-rate reference is always interpolated at 90 %.

Applied to the refinement policy, as implemented, the decision rule of Section 5.5 keeps the teacher-forced curves: on the audited steps, real decoding needs +3.4 % (context memory), +3.6 % (DN-SC)

TABLE 5 | Real channel symbols per token for 90 % token accuracy on the hierarchical drift stream (GPT-2 prior, beam-4 decoder for every receiver). Selective refinement includes its return-link signaling at the entropy; synchronization is not included. Where a curve exceeds 90 % at its cheapest point, the value is read there and is an upper bound (see text). Teacher-forced: mean over seeds, all 600 steps. Decoded for real: 26 audited steps per seed, curves pooled over the seeds, 95 % paired bootstrap interval in brackets. Changes are relative to the frozen fixed-rate receiver on the same seeds.

Receiver	Seeds	Teacher-forced	Change	Decoded for real	Change [95 % CI]	p
Frozen, fixed rate	6	9.23	—	10.37	—	—
Frozen + refinement	6	7.80	−15.4 %	8.36	−19.3 % [−21.4, −15.6]	0.031
Context memory, fixed rate	4	8.04	−10.1 %	9.05	−10.6 % [−16.1, −5.3]	—
Context memory + refinement	4	7.18	−19.8 %	7.55	−25.4 % [−29.0, −15.6]	0.125
DN-SC memories, fixed rate	6	7.80	−15.5 %	8.17	−21.2 % [−23.9, −14.9]	—
DN-SC + refinement	6	**7.08**	**−23.3 %**	**7.48**	**−27.9 %** [−31.7, −21.6]	0.031

p: exact sign-flip test on the per-seed costs decoded for real (refinement receivers against the frozen fixed-rate receiver); Holm-adjusted 0.094, 0.125 and 0.094 for the three rows. Per-seed changes of DN-SC + refinement, seeds 0–5: −23.0, −26.1, −28.2, −10.5, −20.0, −28.5 % (teacher-forced) and −27.9, −30.4, −28.9, −9.1, −29.2, −31.2 % (decoded for real).

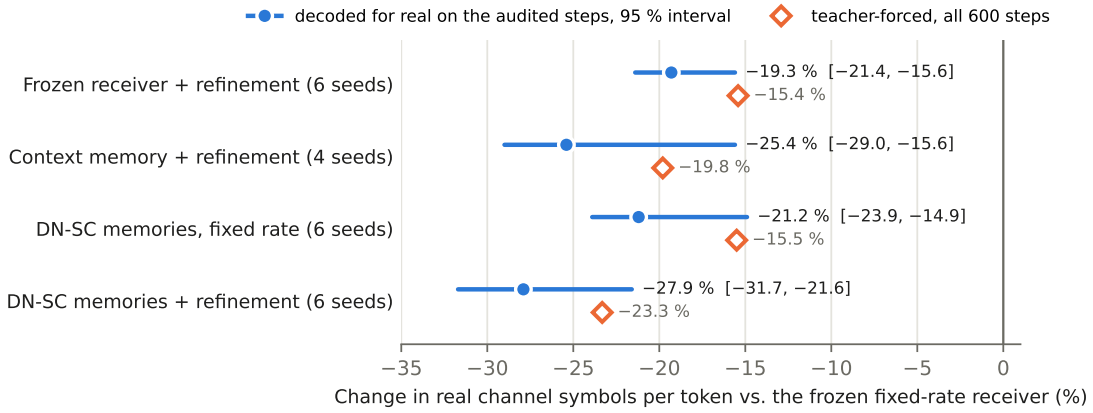


FIGURE 3 | Change in real channel symbols per token for 90 % token accuracy against the frozen fixed-rate receiver, return-link signaling of refinement included. Dots: decoded for real on the audited steps, with 95 % paired bootstrap intervals; diamonds: teacher-forced on all 600 steps.

and +5.0 % (frozen receiver) more forward symbols than the emulation. For the context memory and DN-SC these are readings at the cheapest point, $\tau = 0.5$, at which real decoding is the more accurate of the two. Applied to the fixed-prefix receivers as well, which the wording of the rule also covers, the rule does not keep them: decoded for real, the fixed prefix needs +9.7 % (frozen), +10.9 % (context memory) and +3.8 % (DN-SC) more symbols than emulated on the same steps (10.37 against 9.45 for the frozen reference). Under that stricter reading the real-decoding figures are the measurement. They give the larger savings, because a fixed prefix suffers more from error propagation than the refinement policy does, so the teacher-forced figures quoted as the headline are the conservative ones; both are reported for every receiver. Figure 4 shows the pooled operating curves decoded for real. Refinement moves the curve of the frozen receiver to the left over the whole range it covers (87.7 % accuracy and above); the memories of DN-SC raise the fixed-prefix curve at every rate, which leaves less to refinement.

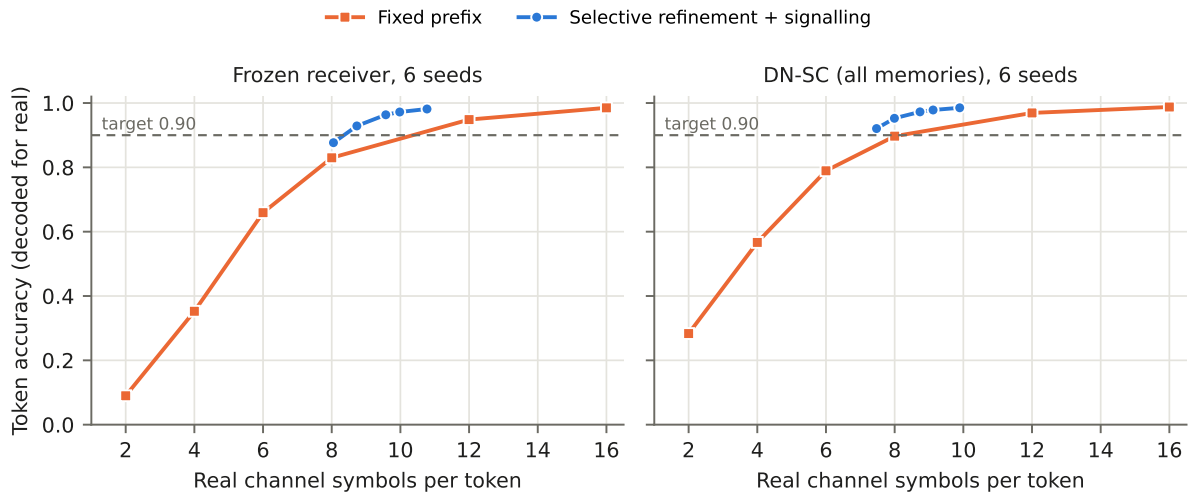


FIGURE 4 | Operating curves decoded for real on the audited steps of the drift stream, pooled over six seeds (156 steps, 936 messages per receiver). Fixed prefix: all six prefix lengths. Selective refinement: thresholds $\tau = 0.5$ to 0.99 from left to right, with the return-link signaling at its entropy added to the forward symbols.

6.4 | Sensitivity to the Power of the Refinement Rounds

The stream runs read every refinement round off one transmission of the message (Section 5.5), so a round that carries a subset of the tokens is sent at the gain of the whole message. To measure the effect, the request policy was emulated on 600 held-out sentences of AG News and TeleQnA (about 10,700 tokens), with the frozen prior and with the true preceding sentences as context, once with that gain and once with every refinement round scaled to unit power over the tokens it carries, as in (6). With the gain of the whole message, the refinement rounds at $\tau = 0.9$ carry on average 1.15 (no context) and 1.12 (context) times the allowed power per symbol, because the tokens that are requested have more energy in the later shells than the average token; individual rounds range from about 0.5 to 2.0 (5th to 95th percentile). Sending them at unit power raises the cost of refinement at 90 % token accuracy by 1.0 % and 0.8 %, and reduces its saving over the fixed prefix from 11.6 % to 10.7 % and from 10.5 % to 9.8 %. If the stream behaved like this text, the cost of DN-SC with refinement would move from 7.08 to about 7.13–7.15 symbols per token and its teacher-forced saving from 23.3 % to about 22.6 %; the fixed-prefix receivers are not affected. The conclusions therefore stand, with savings that are slightly smaller than the stream runs show; the released code sends every refinement round at unit power.

6.5 | Where the Saving Comes From

Receiver memories and selective refinement both contribute, and their contributions overlap (Table 5). At a fixed rate, the memories of DN-SC alone save 15.5 % teacher-forced (21.2 % decoded for real), with no return-link signaling. Refinement alone saves the frozen receiver 15.4 % (19.3 % decoded for real). On top of the memories, refinement adds a further 9.3 % teacher-forced and 8.4 % decoded for real (95 % CI 3.9–18.3 %); since the refinement readings of DN-SC are upper bounds, these are lower bounds of its contribution. The better the prior, the less uncertainty is left for refinement to resolve. Table 6 shows the mechanism. The prior cross-entropy, which measures what the receiver knows about a token before any symbol arrives, falls from 6.79 bits per token for the frozen receiver to 5.61 with the context memory and 5.23 with all memories. On the four seeds shared by all three receivers, the context memory accounts for

TABLE 6 | Mechanism of the saving on the drift stream (teacher-forced, all 600 steps). Prior cross-entropy: mean ± SD over seeds; differences to DN-SC are paired per step, with 95 % block-bootstrap intervals. Requests and rounds at the operating threshold $\tau = 0.9$.

Receiver	Seeds	Prior cross-entropy (bits/token)	Difference to DN-SC [95 % CI]	Requests (bits/token) entropy / bitmap	Rounds per message	Synchronization (kbit/step)
Frozen	6	6.79 ± 0.04	$+1.56$ [$+1.41$, $+1.72$]	1.62 / 1.69	3.07	0
Context memory	4	5.61 ± 0.02	$+0.37$ [$+0.32$, $+0.41$]	1.40 / 1.53	2.81	0
DN-SC	6	5.23 ± 0.02	—	1.39 / 1.52	2.84	24.23

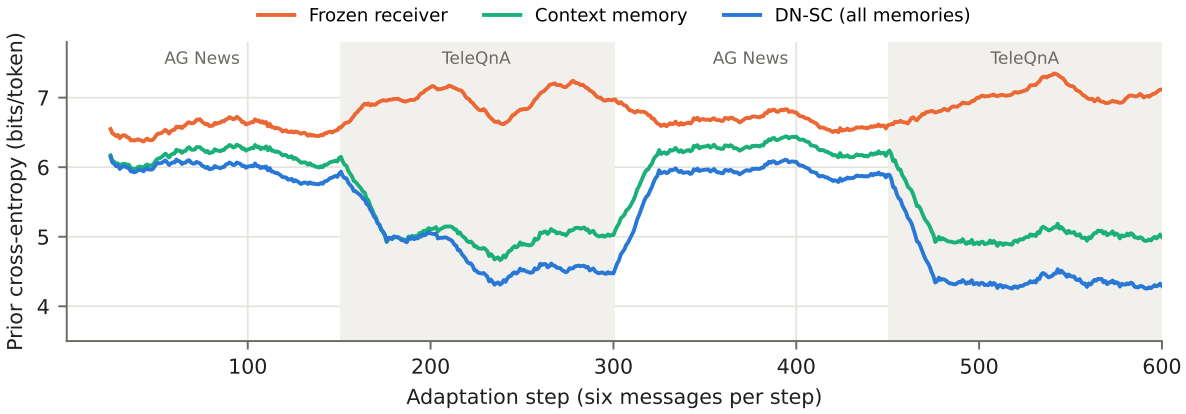


FIGURE 5 | Prior cross-entropy along the hierarchical drift stream (25-step moving average, mean over the four seeds shared by the three receivers). Shaded: TeleQnA regimes.

76 % of that reduction. The shipped memories lower the prior by a further 0.37 bits per token and the selective cost at $\tau = 0.9$ by 0.12 symbols per token; both intervals exclude zero, but with four seeds the seed-level test cannot go below $p = 0.125$. The signaling is light: at the operating threshold the requests cost 1.39 bits per token at the entropy (1.52 as a bitmap) over 2.84 rounds per message for DN-SC, against 1.62 bits and 3.07 rounds for the frozen receiver.

Figure 5 shows the prior along the stream. Within AG News the three receivers differ by less than one bit per token. On the first switch to TeleQnA the frozen prior rises, while the context memory and DN-SC fall to about 5 bits per token within one topic period; a TeleQnA document—question, explanation and answer—repeats much of its own vocabulary, which the context makes available to the prior. The shipped memories add their gain mainly in TeleQnA, and more on its second visit, which is consistent with the recall of the adapter slot learned during the first visit (3.3 slot switches per run on average).

6.6 | Component Analysis

Table 7 reports the components of the memories from the pilot runs, made with an earlier version of the code on a T4 GPU (greedy decoder, first round of 2 symbols, one to three seeds per receiver). They are exploratory, and only the prior cross-entropy is reported, because it does not depend on the decoder of refinement except through the context. The context memory is indispensable: DN-SC without it is 1.14 bits per token worse. Adapter slots help: a single adapter that is dragged between the domains costs 0.10 bits per token. Shipping both gradient levels at one period with about the same synchronization budget (23.73 against 24.23 kbit per step) and one adapter (single-rate) costs 0.11 bits, and adding elastic weight consolidation [34] to it does not help (0.12). The two separate shipping periods, however, are not shown to matter: with slots and context kept and both levels shipped at one period, the difference is 0.02 bits

TABLE 7 | Component analysis (pilot runs, exploratory): prior cross-entropy on the drift stream, mean ± SD over seeds, paired per-step difference to DN-SC with 95 % block-bootstrap interval, equivalence within ±0.05 bits per token by two one-sided tests (90 % interval), and measured synchronization.

Receiver	Seeds	Prior cross-entropy (bits/token)	Difference to DN-SC [95 % CI]	Equivalent (±0.05)	Synchronization (kbit/step)
Frozen	3	6.793 ± 0.044	$+1.537$ [$+1.315$, $+1.760$]	no	0
DN-SC without context	1	6.435	$+1.144$ [$+0.829$, $+1.443$]	no	24.23
Context memory	3	5.628 ± 0.023	$+0.372$ [$+0.322$, $+0.420$]	no	0
Single rate, one adapter	3	5.363 ± 0.024	$+0.107$ [$+0.076$, $+0.143$]	no	23.73
Single rate + EWC	1	5.414	$+0.123$ [$+0.073$, $+0.182$]	no	23.73
DN-SC without slots	3	5.352 ± 0.024	$+0.096$ [$+0.064$, $+0.135$]	no	24.23
DN-SC, one shipping period	1	5.312	$+0.022$ [$+0.013$, $+0.031$]	yes	29.63
DN-SC	3	$\mathbf{5.256 \pm 0.031}$	—	—	24.23
Shipping after every step	3	5.250 ± 0.031	-0.006 [-0.031, $+0.020$]	yes	1185.80

per token, within the equivalence margin of ±0.05, on one seed. Shipping after every step costs 49 times the air time of DN-SC and is equivalent to it within the same margin. On this stream, the gain of the shipped memories therefore comes from the adapter slots and from shipping rarely, not from separating the timescales of the two shipped levels.

6.7 | When Shipping Adaptation Pays

Shipping the updates is not free. DN-SC ships 24.23 kbit per adaptation step; at the capacity of 10 dB this occupies about 14,000 real channel symbols, roughly 110 times the payload of one message at the target accuracy (about 127 symbols). Figure 6 shows the net semantic rate (17) of DN-SC relative to three references, on the four seeds shared by all receivers, with the signaling of refinement included. Against the frozen fixed-rate receiver, shipping pays when one update serves more than about 400 messages, and the saving approaches 21.5 % for large traffic. Against the frozen receiver with refinement, the crossover is about 1250 messages (asymptote 8.1 %), and against the context memory with refinement, which costs no synchronization, it is about 6100 messages, with an asymptotic saving of only 1.8 %. The last comparison is indicative only: on these seeds both refinement readings are mostly upper bounds taken above 90 % accuracy, and they are not taken at equal accuracy. In the stream itself an update serves six messages. For a single link, the context memory with selective refinement is therefore the economical configuration; shipped adaptation pays where one update serves many messages, such as the users of a cell or a multicast group that share a domain.

6.8 | Complexity

The receiver adds three small sets of parameters to the frozen GPT-2 model (124,439,808 parameters): the codebook (213,248, trained once), the medium memory (771 values in the bit ledger, of which 770 are active in this configuration) and one adapter slot (147,456 per slot). On a T4 GPU with one CPU thread, decoding one message of 19 tokens with the language prior took 193–199 ms with greedy decoding, depending on the prefix length, against 3.4–3.7 ms without a prior; the language model dominates the cost. A complete 600-step run of DN-SC, including 26 audited steps decoded for real at five thresholds and six prefix lengths with beam width 4, took 1.45–1.63 h on one thread of the Linux CPU, against 0.99–1.11 h for the frozen receiver. The transmitter must run the language model to learn the receiver's prior, so adaptation costs transmitter computation in addition to air time.

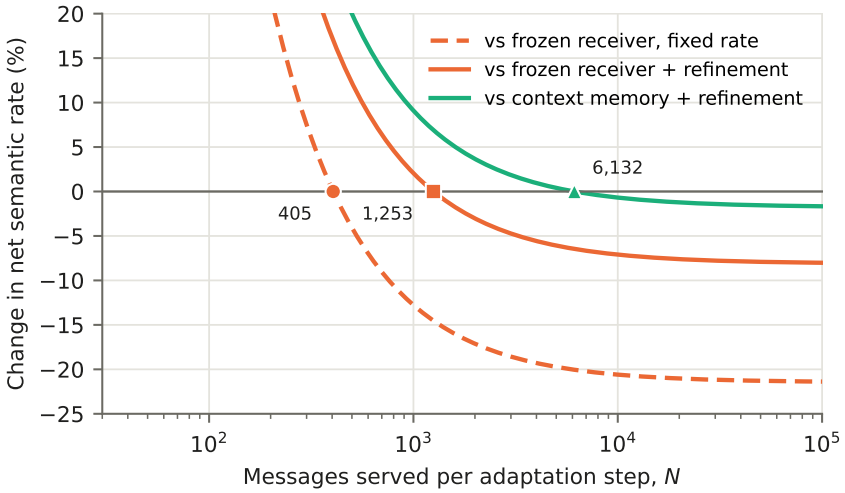


FIGURE 6 | Change in the net semantic rate of DN-SC, synchronization included, relative to three references, against the number of messages one adaptation step serves (four seeds shared by all receivers; return-link signaling at the entropy; synchronization charged at the capacity of 10 dB). Markers and numbers: crossover traffic $N^{\star}$.

6.9 | Discussion and Limitations

What the results support. Every claim in this section is read from result files that the code writes and that are public (Data Availability Statement), on a protocol that was fixed before the confirmatory runs and that allowed the results to go against the method. Three findings are robust: a language prior with selective refinement saves channel symbols against a fixed-rate receiver with the same decoder, signaling included; the free context memory supplies most of what the memories gain; and shipped adaptation pays only above a traffic volume that a single link does not reach. The time-nesting of the shipped memories is the weakest component: the pilot ablation does not show that separating their periods matters, and its measured benefit over the context memory is small.

Position against separate source–channel coding. An idealized separate design that compresses each message by arithmetic coding with the same GPT-2 model and sends it with a finite-block-length channel code in the normal approximation [46] delivers 42 % of the base-corpus messages intact at 4 symbols per token and 96.5 % at 6 (AWGN, 10 dB, known SNR). This is a bound, not a system, but it shows that the uncoded token-level codec does not beat separation at a fixed rate and a known SNR (85.7 % token accuracy at 6 symbols, Table 3). What DN-SC offers instead is graceful degradation without a cliff, a rate that the receiver adapts per token, and an explicit account of what keeping the knowledge aligned costs.

Limitations. (i) One language model (GPT-2 with 124 million parameters) and one codebook were used; the confirmatory runs were made on CPUs, the pilot ablations and the latency measurement on a T4 GPU; larger models will give stronger priors, at a higher cost of computation and of every adapter shipment. (ii) Six seeds give $p = 0.031$ for each comparison of a refinement receiver with the frozen fixed-rate receiver; Holm-adjusted over the three receivers, the smallest value is 0.094. The context memory receiver has four seeds, and its test cannot go below 0.125. (iii) The channel is simulated (AWGN on the stream), with perfect channel state information and a known message length; signaling and synchronization are charged at capacity and assumed error-free, and the delay of several rounds is counted but not modeled. (iv) On the stream, refinement and the receiver's context are emulated

teacher-forced on every step and verified by real decoding on one step in 23; the context is built from the emulated decisions. (v) The component analysis comes from pilot runs of an earlier code version with one to three seeds; it was not repeated with the final decoder. (vi) The stream is text only and designed with hierarchical drift; a stream with naturally occurring drift was not repeated with the final decoder. (vii) Adapter slots are allocated greedily and never recycled, and a token is sent as an uncoded real vector without an outer channel code. (viii) The stream runs sent refinement rounds at the gain of the whole message; on held-out text, sending them at unit power raises the cost of refinement by about 1 % (Section 6.4).

7 | Conclusions and Future Work

This article presented doubly nested semantic communication, a token-level transceiver with a large-language-model prior that is nested in rate and in time, and charged every part of its operation in channel symbols. The rate-nested codebook with per-shell block power control makes every prefix a valid transmission and token-level incremental redundancy exact, at no measurable cost at the prefix that was checked. A Bayesian beam-search receiver requests further symbols only for tokens it is unsure of, and three receiver memories adapt its prior at three timescales, with every shipped bit recorded.

On a hierarchical drift stream with six seeds and a decoder chosen on validation text, DN-SC reached 90 % token accuracy with 7.08 real channel symbols per token against 9.23 for a frozen fixed-rate receiver with the same decoder, a saving of 23.3 % with the return-link signaling charged, observed on every seed, and 27.9 % (95 % CI 21.6–31.7 %) when the audited messages were decoded for real. Most of the gain of the memories came from the conversation context, which costs nothing on the air; the shipped memories lowered the prior by a further 0.37 bits per token, but paid for their synchronization, relative to the context memory with refinement, only when one update served more than about 6100 messages. For a single link, the context memory with selective refinement is the economical design; shipped adaptation becomes worthwhile when one update serves a cell or a multicast group.

Future work will (i) repeat the study with larger language models and on streams with naturally occurring drift; (ii) replace the capacity charge of signaling and synchronization by implemented codes on an error-prone return link, with the delay of the rounds modeled; (iii) share one shipped update among the users of a cell, the regime in which the crossover analysis predicts that adaptation pays; and (iv) derive the shipping period of each memory from its size and drift rate, so that the schedule of the memories becomes a consequence of the cost model rather than a design choice.

Acknowledgments

[To be completed by the authors.]

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The code, the configurations, the decoder selection on validation text, the pre-specified protocol and every result file behind the tables and figures of this article, including the per-step logs of all stream runs, are openly available at `https://github.com/abraheemrashid99-sys/Wiley_paper` (branch `review-fixes`). The primary results are in the folder `results/gpt2_v0.4_six_seeds`; the script `scripts/paper_numbers.py` regenerates every number quoted in the article from those files, and `scripts/run_quick.py` repeats the primary comparison on a laptop CPU. The corpora are public: Europarl (NLTK sample), AG News, TeleQnA and the News Category Dataset.

References

[1] International Telecommunication Union, Radiocommunication Sector, "Framework and Overall Objectives of the Future Development of IMT for 2030 and Beyond," Recommendation ITU-R M.2160-0 (ITU, 2023).

[2] E. C. Strinati and S. Barbarossa, "6G Networks: Beyond Shannon Towards Semantic and Goal-Oriented Communications," *Computer Networks* 190 (2021): 107930.

[3] K. B. Letaief, W. Chen, Y. Shi, J. Zhang, and Y.-J. A. Zhang, "The Roadmap to 6G: AI Empowered Wireless Networks," *IEEE Communications Magazine* 57, no. 8 (2019): 84–90.

[4] W. Saad, M. Bennis, and M. Chen, "A Vision of 6G Wireless Systems: Applications, Trends, Technologies, and Open Research Problems," *IEEE Network* 34, no. 3 (2020): 134–142.

[5] C. E. Shannon, "A Mathematical Theory of Communication," *Bell System Technical Journal* 27, no. 3 (1948): 379–423.

[6] W. Weaver, "Recent Contributions to the Mathematical Theory of Communication," in C. E. Shannon and W. Weaver, *The Mathematical Theory of Communication* (University of Illinois Press, 1949).

[7] D. Gündüz, Z. Qin, I. E. Aguerri, et al., "Beyond Transmitting Bits: Context, Semantics, and Task-Oriented Communications," *IEEE Journal on Selected Areas in Communications* 41, no. 1 (2023): 5–41.

[8] X. Luo, H.-H. Chen, and Q. Guo, "Semantic Communications: Overview, Open Issues, and Future Research Directions," *IEEE Wireless Communications* 29, no. 1 (2022): 210–219.

[9] M. Kountouris and N. Pappas, "Semantics-Empowered Communication for Networked Intelligent Systems," *IEEE Communications Magazine* 59, no. 6 (2021): 96–102.

[10] N. Farsad, M. Rao, and A. Goldsmith, "Deep Learning for Joint Source-Channel Coding of Text," in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (IEEE, 2018), 2326–2330.

[11] H. Xie, Z. Qin, G. Y. Li, and B.-H. Juang, "Deep Learning Enabled Semantic Communication Systems," *IEEE Transactions on Signal Processing* 69 (2021): 2663–2675.

[12] E. Bourtsoulatze, D. Burth Kurka, and D. Gündüz, "Deep Joint Source-Channel Coding for Wireless Image Transmission," *IEEE Transactions on Cognitive Communications and Networking* 5, no. 3 (2019): 567–579.

[13] D. B. Kurka and D. Gündüz, "Bandwidth-Agile Image Transmission With Deep Joint Source-Channel Coding," *IEEE Transactions on Wireless Communications* 20, no. 12 (2021): 8081–8095.

[14] Z. Wang, L. Zou, S. Wei, et al., "Large-Language-Model-Enabled Text Semantic Communication Systems," *Applied Sciences* 15, no. 13 (2025): 7227.

[15] L. Qiao, M. B. Mashhadi, Z. Gao, R. Tafazolli, M. Bennis, and D. Niyato, "Token Communications: A Large Model-Driven Framework for Cross-Modal Context-Aware Semantic Communications," *IEEE Wireless Communications* 32, no. 5 (2025): 80–88.

[16] S. Guo, Y. Wang, S. Li, and N. Saeed, "Semantic Importance-Aware Communications Using Pre-Trained Language Models," *IEEE Communications Letters* 27, no. 9 (2023): 2328–2332.

[17] F. Jiang, L. Dong, Y. Peng, et al., "Large AI Model Empowered Multimodal Semantic Communications," *IEEE Communications Magazine* 63, no. 1 (2025): 76–82.

[18] H. Xie, Z. Qin, X. Tao, and Z. Han, "Toward Intelligent Communications: Large Model Empowered Semantic Communications," *IEEE Communications Magazine* 63, no. 1 (2025): 69–75.

[19] L. Bariah, Q. Zhao, H. Zou, Y. Tian, F. Bader, and M. Debbah, "Large Generative AI Models for Telecom: The Next Big Thing?," *IEEE Communications Magazine* 62 (2024), https://doi.org/10.1109/MCOM.001.2300364.

[20] H. Zhang, S. Shao, M. Tao, X. Bi, and K. B. Letaief, "Deep Learning-Enabled Semantic Communication Systems With Task-Unaware Transmitter and Dynamic Data," *IEEE Journal on Selected Areas in Communications* 41, no. 1 (2023): 170–185.

[21] Y. Zhang, J. Huang, J. Wang, J. Feng, and Z. Fei, "Knowledge Base Identification and Adaptation for End-to-End Semantic Communication," *IEEE Internet of Things Journal* 13, no. 18 (2026): 42698–42712.

[22] J. Zhang, S. Li, Y. Sun, et al., "Cooperative Semantic Knowledge Base Update for Semantic Communication Networks," *IEEE Transactions on Communications* 74 (2026): 1391–1405.

[23] G. Caire and D. Tuninetti, "The Throughput of Hybrid-ARQ Protocols for the Gaussian Collision Channel," *IEEE Transactions on Information Theory* 47, no. 5 (2001): 1971–1988.

[24] P. Jiang, C.-K. Wen, S. Jin, and G. Y. Li, "Deep Source-Channel Coding for Sentence Semantic Transmission With HARQ," *IEEE Transactions on Communications* 70, no. 8 (2022): 5225–5240.

[25] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models Are Unsupervised Multitask Learners," OpenAI Technical Report (2019).

[26] E. J. Hu, Y. Shen, P. Wallis, et al., "LoRA: Low-Rank Adaptation of Large Language Models," in *International Conference on Learning Representations (ICLR)* (2022).

[27] A. Behrouz, M. Razaviyayn, P. Zhong, and V. Mirrokni, "Nested Learning: The Illusion of Deep Learning Architectures," in *Advances in Neural Information Processing Systems 38 (NeurIPS 2025)* (2025), arXiv:2512.24695.

[28] A. Vaswani, N. Shazeer, N. Parmar, et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems 30* (2017), 5998–6008.

[29] P. Koehn, "Europarl: A Parallel Corpus for Statistical Machine Translation," in *Proceedings of Machine Translation Summit X: Papers* (2005), 79–86.

[30] W. H. R. Equitz and T. M. Cover, "Successive Refinement of Information," *IEEE Transactions on Information Theory* 37, no. 2 (1991): 269–275.

[31] A. Kusupati, G. Bhatt, A. Rege, et al., "Matryoshka Representation Learning," in *Advances in Neural Information Processing Systems 35* (2022), 30233–30249.

[32] M. Chen, Z. Sun, X. He, L. Wang, and A. Al-Dulaimi, "LLM-Based Semantic Communication: The Way From Task-Originated to General," *IEEE Wireless Communications Letters* (2025), https://doi.org/10.1109/LWC.2025.3583053.

[33] W. Ni, Z. Qin, Y. Feng, et al., "Multi-Task Semantic Communications via Large Models," *IEEE Communications Standards Magazine* 9, no. 4 (2025): 16–23.

[34] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, et al., "Overcoming Catastrophic Forgetting in Neural Networks," *Proceedings of the National Academy of Sciences of the United States of America* 114, no. 13 (2017): 3521–3526.

[35] D. Alistarh, D. Grubic, J. Li, R. Tomioka, and M. Vojnovic, "QSGD: Communication-Efficient SGD via

Gradient Quantization and Encoding," in *Advances in Neural Information Processing Systems 30* (2017), 1709–1720.

[36] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks From Decentralized Data," in *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, PMLR 54 (2017), 1273–1282.

[37] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* (2002), 311–318.

[38] J. D. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data," in *Proceedings of the 18th International Conference on Machine Learning* (2001), 282–289.

[39] X. Zhang, J. Zhao, and Y. LeCun, "Character-Level Convolutional Networks for Text Classification," in *Advances in Neural Information Processing Systems 28* (2015), 649–657.

[40] A. Maatouk, F. Ayed, N. Piovesan, A. De Domenico, M. Debbah, and Z.-Q. Luo, "TeleQnA: A Benchmark Dataset to Assess Large Language Models Telecommunications Knowledge," *IEEE Network* (2025), arXiv:2310.15051.

[41] R. Misra, "News Category Dataset," arXiv:2209.11429 (2022).

[42] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap* (Chapman & Hall, 1993).

[43] S. Holm, "A Simple Sequentially Rejective Multiple Test Procedure," *Scandinavian Journal of Statistics* 6, no. 2 (1979): 65–70.

[44] H. R. Künsch, "The Jackknife and the Bootstrap for General Stationary Observations," *Annals of Statistics* 17, no. 3 (1989): 1217–1241.

[45] D. J. Schuirmann, "A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability," *Journal of Pharmacokinetics and Biopharmaceutics* 15, no. 6 (1987): 657–680.

[46] Y. Polyanskiy, H. V. Poor, and S. Verdú, "Channel Coding Rate in the Finite Blocklength Regime," *IEEE Transactions on Information Theory* 56, no. 5 (2010): 2307–2359.